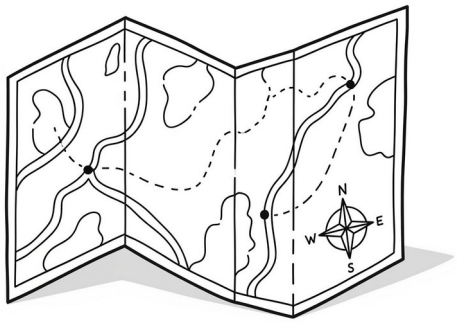


Attempter Guidelines

Change Log

Date	Updates	Notes/Images/Links
Mar 3, 2026	→ Edited Weighting Rules section → Added Golden Review and Rubrics Alignment section	
Feb 27, 2026	→ Updated golden review and rubrics guidelines → Updated Common Errors to Avoid	
Feb 24, 2026	→ Updated the rubric template for **potential fixes**	
Feb 18, 2026	→ Repo is provided. Changes are made as the task artifacts are provided in the task. → Good Prompt Examples section added.	
Feb 2, 2026	→ Initial version.	File

Contents

👋 **Introduction**

📄 **Guidelines**

STEP 1ß Read Task Specs & Constraints

STEP 2ß Draft the Prompt

STEP 3ß Generate Golden Response based on PR

STEP 4ß Generate Rubrics based on the golden response

STEP 5ß Evaluate the Model Evaluation based on Rubrics

📁 **Appendix**

👋 Introduction

Welcome to Menu Territory Code Review! The following document establishes the general expectations for the project. These guidelines apply to all task types.

This project has the following goals:

- → Provide a complex prompt based on the provided task artifacts and create rubrics that challenge cutting-edge AI models.
- → Provide a Golden Response based on the diff.
- → Evaluate a model's ability to identify all issues in a PR, and provide helpful feedback (e.g., propose fixes, generate a final patch) through the creation of Rubrics.
- → Verify the created prompt, and rubrics stumped at least 1 model.

📄 Guidelines

> 💡 **Important Tip**
> Use whatever workflow you prefer to browse PRs and view commit hashes/diffs. The output must be reproducible from public repo data.

Step 1ß Read Task Specs & Constraints	
1ò Repo and other task artifacts provided in the task	Review all the task artifacts (repo, issue title, diff, PR link, etc.) provided in the tasks: ✔️ All required information will be provided to you. ✔️ You are NOT required to perform any harvesting, repository exploration, commit searching, or external lookup.
2ò Save the diff in the prompt	Save the diff exactly as provided. → Copy/paste the provided diff from the task artifact section to the prompt as reference text.

STEP 2ß Draft the Prompt

The prompt should assign a persona of a senior software engineer (or equivalent) tasked with performing a code review. A review of the PR must be clearly requested.

The ñDiffò reference text of the prompt must have issues, so that there is something to point out in the code review. **There must be at least one serious blocking issue in the given diff.** If there are no serious issues in the given diff, please flag the task and skip the task. Here is a list of the issues that are allowed:

Serious Blocking Issues	Non-serious Issues
→ new bugs/correctness → security risk → functionality failures → missing core tests → core edge cases → performance regression → API misuse → Etc	→ poor formatting → dependency bumps → subpar documentation → Etc

To craft your prompt, make sure to give the following structure:	
Role + goal ñ2ù3 sentences)	"You are a senior software engineer reviewing a PR diff to verify it solves the problem statement, is production-ready, and introduces no bugs."
Problem statement ñ3ù8 lines)	Explain what's broken / what's expected. Keep it concrete.
Diff saved	The diff saved as reference text in the prompt should contain the entire diff from the task artifacts including all file changes.
Prompt Type/Output requirements (pick ONEò	**Review-only**: "Return a detailed PR review with issues, risks, and required changes before merge." **Review + fix** ñAirflow-style): "If issues exist, propose a fix plan and then upload a corrected patch in your golden response."

STEP 3ß Generate Golden Response based on PR

Write the Golden Review as the ideal senior-engineer PR review comment. It must be structured, specific, and actionable, and it must match the task's required output (review-only vs review+fix).

The Golden Review should encapsulate what an ideal response should look like — **it shouldn't fail any of the Rubric's criteria**, and **should be free of any objective inaccuracies**, and its **code changes should not introduce errors**.

Required Structure	
1. Summary ñ2ø4 sentences)	→ Does the patch solve the problem statement? → What are the top 1ù2 issues that matter most?
2. Blockers (must-fix before merge)	→ Bullet list → For each blocker, include: • What's wrong • Why it matters (bug, regression risk, correctness, UX, etc.) • Exact fix (**conceptual solutions** on what to change) • Where (file path + line range if available in the diff) → Each task must include at least one blocker
3. Non-blocking suggestions	→ Optional improvements (readability, small cleanup). → Keep it short and avoid scope creep.
4. Tests / Validation	→ List the specific edge cases that must be tested. → Say what should fail before the fix and pass after it. → If the change affects behavior, require a regression test.
5. Fix plan ñONLY if the task requires a plan)	→ 3ù6 steps max. → Each step must map to a blocker.
6. Corrected patch ñONLY if the task requires providing a patch)	→ Upload the correct patch file with a golden response. → **A golden diff without any blockers** and keep changes minimal (no unrelated refactors).

STEP 4ß Generate Rubrics based on the golden response

Create a **minimum of 15** rubric items that cover:

1. **Core correctness** (must-catch bug/issue)
   The response identifies the actual problem and why it breaks expected behavior.
   ñAirflow example: must note. also needs stripping, not only -.)

2. **Required fix**
   The response proposes the correct change(s) that fully solve the stated problem.
3. **Regression/edge-case coverage**
   The response asks for or provides tests that cover the critical edge cases.
4. **Scope discipline**
   The response does not suggest unrelated refactorings or changes outside the problem.
5. **Clarity / actionable review**
   The response is structured and gives clear, actionable feedback (what to change + where + why).
6. **Hygiene checks when applicable**
   The response flags obvious issues like debug prints, conflicting CLI flags, missing prompts/required args, etc. ñSnowflake sample themes.)

The rubrics for specific or explicit requests in the prompt should avoid, at all costs, being subjective, too vague, overly broad, or unmeasurable (e.g., “the response should have good formatting” or “code must be optimal”), and they should also cover every single serious issue in the PR.

These rubrics should never check for something that contradicts what's explicitly required in the prompt, contain a factual error or misleading point, or check something that is not either an explicit or implicit requirement from the prompt and including it in the Golden Review does not improve anything.

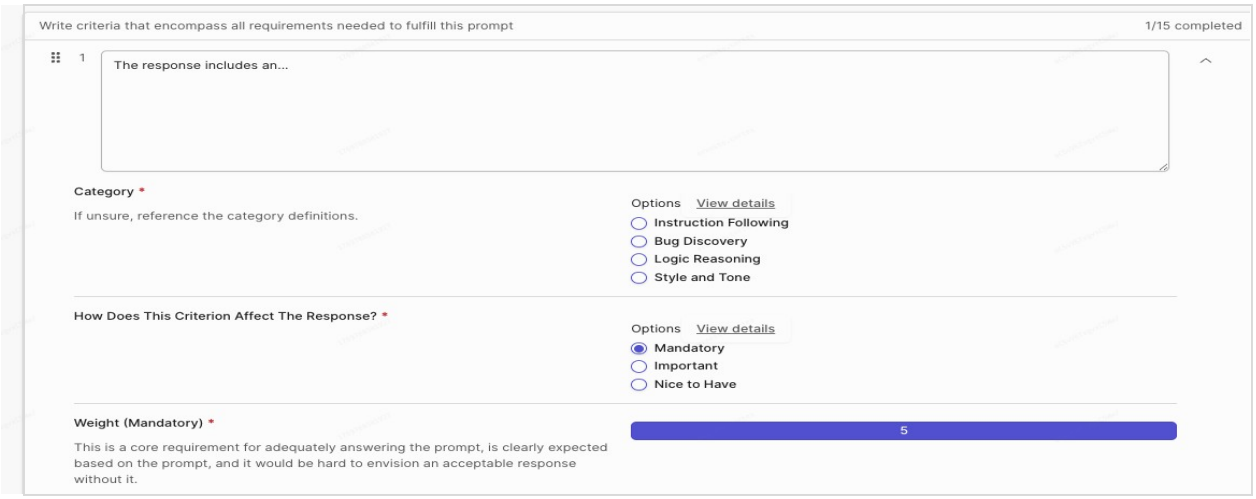
Please ensure there are no missing criteria: all blocker issues, as well as non-blocking suggestions and tests/validation, must have corresponding criteria.

When writing rubrics for potential fixes, focus on describing necessary modifications and proposing conceptual solutions similar to a human review rather than strictly requiring exact corrected code; please ensure it is evaluated using a generalized rubric item "The response suggests a fix for <blocker_x>, for example <suggested_fix>" to maintain flexibility and accommodate multiple valid implementation methods.

Finally, all the rubrics should be self-contained. Ask yourself: “Using **only the rubrics, and the Model Response**, am I able to tell that the Model Response is correct?” If yes, you can include that rubric. But if not, and you think that you should access to external info to determine if the response is correct or not, then that rubric is not self-contained and it should not be included.

Rubric Types	
Instruction Following	Ensures the response adheres to explicit directions in the prompt (formats, constraints, language, libraries, required elements).
Bug Discovery	Verifies the model identifies and suggests solutions for bugs or issues present in the PR.
Logic Reasoning	Verifies the model identifies and suggests solutions for any reasoning flaws.
Style and Tone	**NICE TO HAVE ONLY** - Checks for any formatting or tone requirements that would make the response better looking.

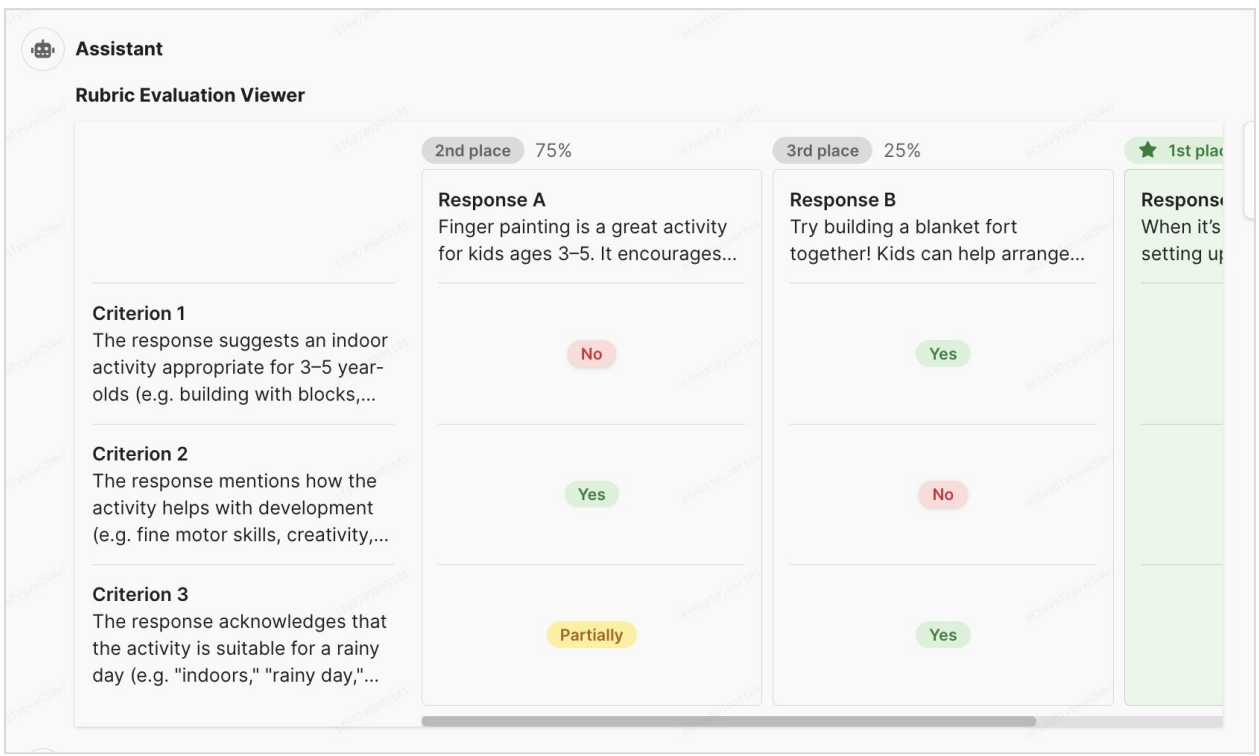
Weighting Rules Each rubric criterion must have one of the following weights:	
5 — Mandatory	- Core requirement for adequately answering the prompt. - Response would be unacceptable without it. - Explicit requirement from the prompt. - Recognition or identification of serious blocking issues. - Possible solution or fix for new bugs/correctness issues, functionality failures, API misuse, security risks, missing core tests, performance regression, core edge cases, etc.
3 — Important	- Makes the response substantially better. - Implicit requirement from the prompt. - Response would be acceptable without it if all other criteria are met. - Recognition of non-blocking issues and suggestions, tests/validations.
1 — Nice to have ñOptional)	- Nice-to-have and necessary for a perfect response. - Response can still be strong enough without it. - Could include best code practices, libraries, language versions, formatting, style/tone etc which are not asked in the prompt explicitly.

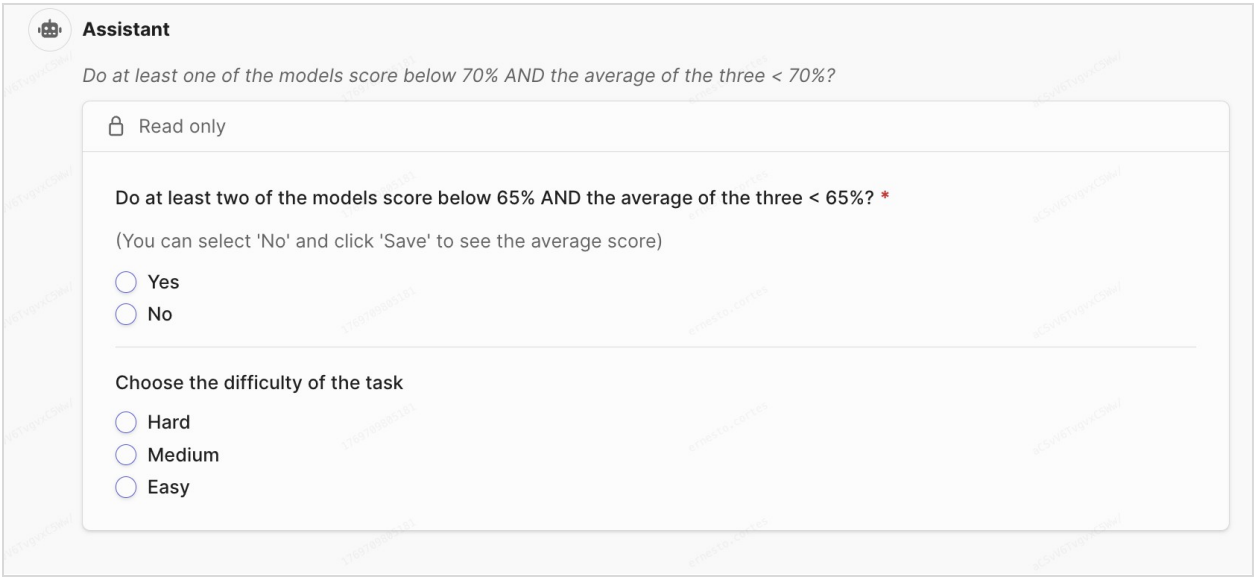


STEP 5ß Evaluate the Model Evaluation based on Rubrics

Does at least one of the models score below 70% AND the average of the three < 70%?

- **If yes**, you can proceed with labeling and finishing the task
- **If not**, you need to tweak the rubrics and/or prompt to increase the complexity





📌 Golden Review Response and Rubrics Alignment

🚨 Remember, the golden review you have written should pass all the rubric criteria that you have created, including Mandatory criteria, Important criteria, and Nice to Have criteria. Your task will be considered a failure and rejected if the golden review fails even one of the rubric's óMandatory or Important] criteria, and it will receive a maximum review score of only 3 if it fails even one [**Nice to Have**] criterion.

Please give a special focus to align your golden review, prompt, and rubrics based on the **unified diff provided in the task**.

Helpful Tips:

- Always make sure the golden review response fulfills the prompt request completely.
- Always make sure the golden review response and the prompt is aligned with the unified diff.
- While creating rubrics, if you add criteria that are not covered or fulfilled by the golden review, go back to your golden review and make sure that it passes the new criteria you have added.

✅ Good Prompt Examples

Some examples of good prompts for this project are provided below. Use them as references but do not try to use them directly in your task.

None

You are a senior software engineer reviewing a pull request diff in the Healthchecks repository. Healthchecks is a production cron job monitoring service built with Python 3.12, Django, and a relational database (PostgreSQL, MySQL, or MariaDB). Your role is to verify that the changes correctly solve the stated problem, are production-ready, and do not introduce regressions or subtle bugs.

Healthchecks relies on incoming “ping” requests to track the execution of scheduled tasks. Currently, the system cannot identify individual executions of overlapping or retried jobs, which leads to incorrect duration calculations and duplicated ping records. When clients retry failed requests, multiple identical pings can be logged, breaking idempotence and inflating execution history. Users running overlapping cron jobs cannot reliably track the start and end times of each run, and the check state can become inconsistent when one run finishes while another is still active.

This PR introduces support for a user-specified “run id” parameter to enable accurate tracking of overlapping runs and to lay groundwork for making ping requests idempotent.

A full unified diff is attached as a reference text Review the diff as the sole source of code changes.

Output format:

1) Summary

2) Blockers (for each, state what’s wrong, why it matters, how to fix it, where it exists)

3) Non-blocking suggestions

4) Tests/validation

Prompt #2ß

None

You are a senior software engineer reviewing a pull request diff for PyOD (Python Outlier Detection library). Your role is to verify the changes solve the problem statement, are production-ready, follow PyOD conventions, and do not introduce regressions, runtime errors, or subtle bugs. Your review must cover correctness, performance/scalability, API consistency, and edge case handling.

This PR adds dimensional explainability to the KNN outlier detector in PyOD. The goal is to provide:

- get_outlier_explainability_scores(): returns per-feature outlier contribution scores as a NumPy array

- explain_outlier(): visualizes these scores via horizontal bar charts with percentile cutoff bands

Ensure consistency with COPOD's existing explain_outlier() API pattern in PyOD.

The implementation should follow PyOD conventions for new detector features, handle edge cases properly, avoid breaking existing functionality, and maintain performance characteristics for large datasets. The PR modifies pyod/models/knn.py, adds a new example file, and includes unit tests.

A full unified diff is attached as a reference text. Review the diff as the sole source of code changes.

Return a detailed PR review with issues, risks, and required changes before merge.

Output format:

1) Summary

2) Blockers (for each, state what's wrong, why it matters, how to fix it and where it exsists)

3) Non-blocking suggestions

4) Tests/validation

❌ Common Errors to Avoid

1. Rubrics are not self-contained.

Some rubrics require referencing the original prompt to evaluate the quality of the model's response. However, during evaluation, we only have access to the model's response. Rubrics must therefore be fully self-contained and allow assessment based solely on the response itself.

❌ **The Bad Example**

```
None
```

The response verifies that clear, actionable feedback is given by specifying what to change, and why, without hallucinating files that are not in the diff.

Why this fails:

When evaluating the model using this criterion, we do not have access to the diff, only the model's response. Imagine that you only have access to the model response and are trying to evaluate if it fulfilled the rubric item. Will you be able to evaluate accurately without referencing anything else?

2. Blockers are based on assumptions rather than concrete code behavior.
   A blocker should be grounded in the actual implementation of the provided Diff file and observable code logic in the Diff, not on speculative or hypothetical scenarios.

3. Fix-related rubrics are overly restrictive.
   Some rubrics exclude alternative valid solutions by prescribing a single specific fix. To maintain flexibility and accommodate multiple correct implementation approaches, please use the following format:
   "The response suggests a fix for <blocker_x>, for example <suggested_fix>."

❌ **The Bad Example**

```
None
```

The response suggests removing the inspector widget from the splitter before calling `deleteLater()` in the `set_position()` method in `qutebrowser/browser/inspector.py`.

Why this fails:

The overfitting criterion for suggested fixes for the blockers is too restrictive - it allows only one valid fix suggestion instead of allowing for multiple solutions. Any method that ensures the widget is detached from the splitter before deletion is valid

✅ The Good Example

None

The response suggests a fix for Blocker: the Inspector widget is not removed from splitter when closed, for example, removing the inspector widget from the splitter before calling `deleteLater()` in the `set_position()` method in `qutebrowser/browser/inspector.py`.

4. Rubrics cannot be satisfied by the provided Golden Response.

   All rubrics must be passable by the Golden Response. Please ensure that the Golden Response represents the optimal solution. Even “nice-to-have” rubrics should be satisfied by it.

✅ Good Task Examples

⚠️Please do not copy and paste ANY information from the tasks.
⚠️Use them only as a reference.

🧷Example #1

None

Task Metadata:

Github Repo
https://github.com/keycloak/keycloak

Repository Name and Description
keycloak

Keycloak: Open Source Identity and Access Management For Modern Applications and Services

Base Commit

8af4a5fc852b2aeba8617ef71cf86573fe28b27f

Target Commit

d0a93726882a35f76024dcdaeff9ef67f9081eff

PR Link

https://github.com/keycloak/keycloak/pull/7035

Issue Title

[KEYCLOAK-14046] Include groups in user creation via Admin Console

Diff:

```
diff --git a/themes/src/main/resources/theme/base/admin/messages/admin-messages_en.properties
b/themes/src/main/resources/theme/base/admin/messages/admin-messages_en.properties
index 18e913f724..c7d073858e 100644
--- a/themes/src/main/resources/theme/base/admin/messages/admin-messages_en.properties
+++ b/themes/src/main/resources/theme/base/admin/messages/admin-messages_en.properties
@@ -1394,6 +1394,11 @@ unlock-user=Unlock user
 federation-link=Federation Link
 email-verified=Email Verified
 email-verified.tooltip=Has the user's email been verified?
+groups-joining=Groups
+groups-joining.tooltip=Groups the user will be joining. To add a group, search for any existing one
and select it.
+groups-joining-select.placeholder=Select existing group
+groups-joining-no-selected=No group selected
+groups-joining-path=Path
 required-user-actions=Required User Actions
 required-user-actions.tooltip=Require an action when the user logs in. 'Verify email' sends an email
to the user to verify their email address. 'Update profile' requires user to enter in new personal
information. 'Update password' requires user to enter in a new password. 'Configure OTP' requires
setup of a mobile password generator.
 locale=Locale
diff --git a/themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js
b/themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js
index 7d7e4d0d3e..34df0290c3 100755
--- a/themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js
+++ b/themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js
@@ -385,14 +385,15 @@ module.controller('UserDetailCtrl', function($scope, realm, user, BruteForceUser
                                 Components,
                                 UserImpersonation, RequiredActions,
```

```
                                        UserStorageOperations,
-                                       $location, $http, Dialog, Notifications, $translate) {
+                                        $location, $http, Dialog, Notifications, $translate,
Groups) {
    $scope.realm = realm;
    $scope.create = !user.id;
    $scope.editUsername = $scope.create || $scope.realm.editUsernameAllowed;
    $scope.emailAsUsername = $scope.realm.registrationEmailAsUsername;
+   $scope.groupSearch = { selectedGroup : null };

    if ($scope.create) {
-       $scope.user = { enabled: true, attributes: {} }
+       $scope.user = { enabled: true, attributes: {}, groups: [] }
    } else {
        if (!user.attributes) {
            user.attributes = {}
@@ -464,6 +465,8 @@ module.controller('UserDetailCtrl', function($scope, realm, user, BruteForceUser
        convertAttributeValuesToLists();

        if ($scope.create) {
+           pushSelectedGroupsToUser();
+
            User.save({
                realm: realm.realm
            }, $scope.user, function (data, headers) {
@@ -513,6 +516,51 @@ module.controller('UserDetailCtrl', function($scope, realm, user, BruteForceUser
        }
    }

+   function pushSelectedGroupsToUser() {
+       var groups = $scope.user.groups;
+       if ($scope.selectedGroups) {
+           for (i = 0; i < $scope.selectedGroups.length; i++) {
+               var groupPath = $scope.selectedGroups[i].path;
+               if (!groups.includes(groupPath)) {
+                   groups.push(groupPath);
+               }
+           }
+       }
+   }
+
+   function bfs(tree, collection) {
+               if (!tree["subGroups"] || tree["subGroups"].length === 0) return;
+               for (var i=0; i < tree["subGroups"].length; i++) {
```

```
+                                var child = tree["subGroups"][i]
+                                collection.push(child);
+                                bfs(child, collection);
+                }
+                return;
+    }
+
+    function flattenGroups(groups) {
+        var flattenedGroups = [];
+        if (!groups || groups.length === 0) return groups;
+        for (var i=0; i < groups.length; i++) {
+            flattenedGroups.push(groups[i]);
+            bfs(groups[i], flattenedGroups);
+        }
+
+        return flattenedGroups;
+    }
+
+    /**
+     * Only keep groups that :
+     *   - include the search term in their path
+     *   - are not already selected
+     */
+    function filterSearchedGroups(groups, term, selectedGroups) {
+        if (!groups || groups.length === 0) return groups;
+        if (!selectedGroups) selectedGroups = [];
+
+        return groups.filter(group => group.path?.includes(term) && !selectedGroups.some(selGroup =>
selGroup.id === group.id));
+    }
+
     $scope.reset = function() {
         $scope.user = angular.copy(user);
         $scope.changed = false;
@@ -530,6 +578,67 @@ module.controller('UserDetailCtrl', function($scope, realm, user, BruteForceUser
     $scope.removeAttribute = function(key) {
         delete $scope.user.attributes[key];
     }
+
+    $scope.groupsUiSelect = {
+        minimumInputLength: 1,
+        delay: 500,
+        allowClear: true,
+        query: function (query) {
```

```
+            var data = {results: []};
+            if ('' == query.term.trim()) {
+                query.callback(data);
+                return;
+            }
+            $scope.query = {
+                realm: realm.realm,
+                search: query.term.trim(),
+                max : 20,
+                first : 0
+            };
+            Groups.query($scope.query, function(response) {
+                data.results = filterSearchedGroups(flattenGroups(response), query.term.trim(),
$scope.selectedGroups);
+                query.callback(data);
+            });
+        },
+        formatResult: function(object, container, query) {
+            object.text = object.path;
+            return object.path;
+        }
+    };
+
+    $scope.removeGroup = function(list, group) {
+        for (i = 0; i < angular.copy(list).length; i++) {
+            if (group.id == list[i].id) {
+                list.splice(i, 1);
+            }
+        }
+    }
+
+    $scope.selectGroup = function(group) {
+        if (!group || !group.id) {
+            return;
+        }
+
+        $scope.groupSearch.selectedGroup = group;
+
+        if (!$scope.selectedGroups) {
+            $scope.selectedGroups = [];
+        }
+
+        for (i = 0; i < $scope.selectedGroups.length; i++) {
+            if ($scope.selectedGroups[i].id == group.id) {
```

```
+                return;
+            }
+        }
+
+        $scope.selectedGroups.push(group);
+        $scope.groupSearch.selectedGroup = null;
+    }
+
+    $scope.clearGroupSelection = function() {
+        $scope.groupSearch.selectedGroup = null;
+        $('#groups').val(null).trigger('change.select2');
+    }
 });

 module.controller('UserCredentialsCtrl', function($scope, realm, user, $route, $location,
RequiredActions, User, UserExecuteActionsEmail,
diff --git a/themes/src/main/resources/theme/base/admin/resources/partials/group-list.html
b/themes/src/main/resources/theme/base/admin/resources/partials/group-list.html
index 731e28d42c..3412e6a2d9 100755
--- a/themes/src/main/resources/theme/base/admin/resources/partials/group-list.html
+++ b/themes/src/main/resources/theme/base/admin/resources/partials/group-list.html
@@ -1,7 +1,7 @@
 <div class="col-sm-9 col-md-10 col-sm-push-3 col-md-push-2">
     <kc-tabs-group-list></kc-tabs-group-list>

-    <table class="table table-striped table-bordered" style="margin-bottom: 0">
+    <table class="table table-striped table-bordered" id="group-table" style="margin-bottom: 0">
         <thead>
         <tr>
             <th class="kc-table-actions" colspan="5">
diff --git a/themes/src/main/resources/theme/base/admin/resources/partials/user-detail.html
b/themes/src/main/resources/theme/base/admin/resources/partials/user-detail.html
index 31583774fa..d7b4623886 100755
--- a/themes/src/main/resources/theme/base/admin/resources/partials/user-detail.html
+++ b/themes/src/main/resources/theme/base/admin/resources/partials/user-detail.html
@@ -97,6 +97,42 @@
                 </div>
                 <kc-tooltip>{{:: 'email-verified.tooltip' | translate}}</kc-tooltip>
             </div>
+            <div class="form-group clearfix block" data-ng-if="create">
+                <label class="col-md-2 control-label" for="selected-groups">{{:: 'groups-joining' |
translate}}</label>
+                <div class="col-sm-6">
+                    <table class="table table-striped table-bordered" style="margin-top: 0px"
id="selected-groups">
```

```
+                    <thead>
+                    <tr>
+                        <th class="kc-table-actions" colspan="2">
+                            <div class="form-inline col-md-12 select2-container-single">
+                                <div class="form-group">
+                                    <div class="input-group">
+                                        <input type="hidden" ui-select2="groupsUiSelect"
id="groups" data-ng-change="selectGroup(groupSearch.selectedGroup);"
data-ng-model="groupSearch.selectedGroup" data-placeholder="{{:: 'groups-joining-select.placeholder' |
translate}}..."/>
+                                    </div>
+                                </div>
+                            </div>
+                        </th>
+                    </tr>
+                    <tr data-ng-hide="!selectedGroups || selectedGroups.length == 0">
+                        <th>{{:: 'groups-joining-path' | translate}}</th>
+                        <th width="20%">{{:: 'actions' | translate}}</th>
+                    </tr>
+                    </thead>
+                    <tbody>
+                    <tr ng-repeat="group in selectedGroups">
+                        <td>{{group.path}}</td>
+                        <td class="kc-action-cell" ng-click="removeGroup(selectedGroups, group);"
style="vertical-align: middle">
+                            {{:: 'remove' | translate}}
+                        </td>
+                    </tr>
+                    <tr data-ng-show="!selectedGroups || selectedGroups.length == 0">
+                        <td class="text-muted" colspan="2">{{:: 'groups-joining-no-selected' |
translate}}</td>
+                    </tr>
+                    </tbody>
+                </table>
+            </div>
+            <kc-tooltip>{{:: 'groups-joining.tooltip' | translate}}</kc-tooltip>
+        </div>
        <div class="form-group clearfix">
            <label class="col-md-2 control-label" for="reqActions">{{:: 'required-user-actions' |
translate}}</label>

diff --git a/themes/src/main/resources/theme/keycloak/admin/resources/css/styles.css
b/themes/src/main/resources/theme/keycloak/admin/resources/css/styles.css
index 681b3fc045..5fedbddda5 100755
--- a/themes/src/main/resources/theme/keycloak/admin/resources/css/styles.css
+++ b/themes/src/main/resources/theme/keycloak/admin/resources/css/styles.css
```

```
@@ -190,6 +190,20 @@ th.w-40 {
    height: 26px;
 }

+.select2-container-single {
+    padding: 0;
+    margin: 0;
+}
+
+.select2-container-single .form-group {
+    width: 100%;
+    margin: 0;
+}
+
+.select2-container-single .form-group .input-group {
+    width: 100%;
+}
+
 /*********** html select ********/
 .overflow-select {
    overflow: auto;
```

User Prompt:

You are a senior software engineer reviewing a Pull Request diff in the Keycloak repository. Keycloak is an open-source Identity and Access Management platform that provides authentication, user federation, authorization, and administrative tooling. The Admin Console is a critical interface for managing users, roles, and groups, so UI changes must remain consistent with the underlying REST API behavior and not introduce security or state inconsistencies.

Your role is to verify that the changes correctly solve the stated problem, are production-ready, and do not introduce any bugs or issues.

Keycloak's POST /users Admin REST API now supports specifying group membership during user creation. The goal of this PR is to update the Admin Console UI to allow administrators to assign groups when creating a new user, aligning the UI capabilities with the API. This involves adding a new field in the user creation form to select groups and wiring it correctly into the user creation request payload, as well as adding the necessary localized messages.

I provided the diff as a reference text below. Provide the detailed diff review with the following structure:

Summary

Blockers (for each blocker, include what is wrong, why it matters, how to fix it and where it exists in the diff)

Non-blocking suggestions

Tests/Validation

Golden response:

1. Summary

This patch adds a group selection field to the Admin Console user creation form, allowing administrators to assign groups at the time a new user is created. The overall approach is correct and aligns the UI with the existing `POST /users` API capability. However, there are two issues that must be addressed before merge: the Select2 input widget is not visually reset after a group is selected (causing a clear UX regression), and the loop counter variables are implicitly global across three functions.

2. Blockers

- Blocker 1: Select2 input does not reset after group selection.

Why it matters:

`$scope.selectGroup()` sets `$scope.groupSearch.selectedGroup = null` after pushing the group to `$scope.selectedGroups`. This updates the Angular model, but Select2 maintains its own internal DOM state independently. Setting the angular model to `null` does not cause Select2 to visually clear its input field. As a result, the selected group's name remains displayed in the search box even after it has been added to the table, making the UI appear broken to the admin.

Why it matters:

A `$scope.clearGroupSelection()` helper already exists in the same controller for exactly this purpose: it resets both the Angular model and fires the necessary Select2 event (`$('#groups').val(null).trigger('change.select2')`), but it is never called from `$scope.selectGroup()`. This is a functional UX regression that is visible on every group selection.

Exact fix:

At the end of `$scope.selectGroup()`, replace the direct assignment `$scope.groupSearch.selectedGroup = null` with a call to the existing `$scope.clearGroupSelection()` helper instead. This ensures both the Angular model and the Select2 widget state are reset togheter.

Where:

The `$scope.selectGroup` function in the `themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js` file (diff lines 157-158).

- Blocker 2: Implicit global variable `i` in three functions.

What's wrong:

The loop counter `i` is used without `var` in `pushSelectedGroupsToUser()`, `$scope.removeGroup()`, and `$scope.selectGroup()`. In non-strict JavaScript, an undeclared variable assignment creates a property on the global `window` object.

Why it matters:

All three functions share the same `global.i`. If any two of these functions are active at the same time (for example, if Angular's digest cycle triggers `selectGroup` while `removeGroup` is mid-loop, or if `pushSelectedGroupsToUser` is called during form submission while a concurrent watch resolves), they will overwrite each other's loop counter. This leads to incorrect iteration: elements being skipped, iterated multiple times, or array access going out of bounds. This creates a real bug, and is not just a style concern.

Exact fix:

Declare `var i` explicitly in each of the three for-loop initializers: in `pushSelectedGroupsToUser()`, `$scope.removeGroup()`, and `$scope.selectGroup()`. This scopes the variable to the function rather than the global object.

Where:

`themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js` on diff lines 55, 133, 151.

3. Non-blocking suggestions

- `Groups.query()` has no error callback: if the API call fails, the search silently returns no results with no user feedback. Add an error callback that surfaces a notification to the user.

4. Tests/Validation

The following tests should be performed to ensure that the patch works as expected:

- Select2 reset: select a group from the dropdown. Before the fix, the input remains populated with the selected group's name after it is added to the table. After the fix, the input clears back to the placeholder immediately.
- Duplicate prevention: attempt to select the same group twice. The selected groups table must not contain duplicate entries on either attempt.
- Whitespace only search input: Type only spaces in the search field. No API call should be triggered and the dropdown should not open.

Rubric:

Criteria	Rubric Type	Rubric Weights Mandatory = 5 Important = 3 Nice to Have = 1
The response includes a section titled "Summary".	Instruction following	mandatory
The response includes a section titled "Blockers".	Instruction following	mandatory
The response includes a section titled "Non-blocking suggestions".	Instruction following	mandatory

The response includes a section titled "Tests/Validation".	Instruction following	mandatory
The response identifies that `$scope.selectGroup()` in `themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js` does not visually reset the Select 2 input widget after a group is added to the selection list.	bug discovery	mandatory
The response suggests a fix for the Select2 widget not being reset, for example, calling `$scope.clearGroupSelection()` inside `themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js` instead of only setting `$scope.groupSearch.selectedGroup = null`.	bug discovery	mandatory
The response identifies that the loop variable `i` is used without a `var` declaration in the `pushSelectedGroupsToUser()` function in `themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js`, making it an implicit global variable that can be overwritten by any other function using an undeclared `i` concurrently.	bug discovery	mandatory
The response identifies that the loop variable `i` is used without a `var` declaration in the `$scope.removeGroup()` function in `themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js`, making it an implicit global that can be overwritten by any other function using an undeclared `i` concurrently.	bug discovery	mandatory
The response identifies that the loop variable `i` is used without a `var` declaration in the `$scope.selectGroup()` function in `themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js`, making it an implicit global variable that can be overwritten by any other function using an undeclared `i` concurrently.	bug discovery	mandatory

the response recommends declaring `var i` explicitly in the for-loop initializer of the `pushSelectedGroupsToUser()` function in `themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js`.	bug discovery	mandatory
The response recommends declaring `var i` explicitly in the for-loop initializer of the `$scope.removeGroup()` function in `themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js`.	bug discovery	mandatory
The response recommends declaring `var i` explicitly in the for-loop initializer of the `$scope.selectGroup()` function in `themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js`.	bug discovery	mandatory
The response recommends performing a test to verify that the Select2 input clears back to the placeholder after a group is selected and added to the list.	instruction following	mandatory
The response identifies that the `Groups.query()` call inside the `groupsUiSelect` query function in `themes/src/main/resources/theme/base/admin/resources/js/controllers/users.js` has no error callback, causing API failures to produce no user visible feedback.	bug discovery	important
The response recommends performing a test that verifies that attempting to add the same group twice does not result in duplicate entries in the selected group list.	instruction following	important
The response recommends performing a test that verifies that entering only whitespace in the group search field does not trigger an API call.	instruction following	important